

Gợi mở từ bài toán Sokoban

Liu Rujia

Nguồn gốc: Insights from the Sokoban problem

IOI China candidate team papers / 2001 / Liu Rujia (incomplete).pdf

Từ khóa. Bài toán Sokoban; tìm kiếm trí tuệ nhân tạo; IDA*.

Tóm tắt

Bài viết này thảo luận một bài toán vừa thú vị vừa đầy thách thức: bài toán Sokoban. Bài viết bắt đầu từ các kiến thức cơ bản của tìm kiếm trong không gian trạng thái, chọn thuật toán IDA* theo đặc điểm của bài toán Sokoban, rồi thực hiện một số cải tiến bước đầu. Phần chính của bài viết bàn về vài cách làm cho chương trình thông minh hơn: cải thiện ước lượng cận dưới, nhận biết bế tắc, phân rã và hợp nhất nhiệm vụ hợp lý, tìm kiếm mẫu và thử nghiệm ngẫu nhiên. Cuối cùng, bài viết giới thiệu sơ lược một số ý tưởng khác chưa được nhắc tới ở trước và đưa ra tổng kết.

Mục lục

Lời nói đầu	2
1 Kiến thức cơ bản về tìm kiếm không gian trạng thái	2
1.1 Không gian trạng thái	2
1.2 Tìm kiếm mù	2
1.3 Tìm kiếm có thông tin	3
2 Bài toán Sokoban và đặc điểm của nó	5
2.1 Bài toán Sokoban	5
2.2 Đặc điểm của bài toán Sokoban	5
2.3 Giải bài toán Sokoban như thế nào	6
3 Dùng thuật toán IDA* giải bài toán Sokoban: hiện thực và cải tiến	6
3.1 Khung thuật toán IDA*	6
3.2 Chương trình đầu tiên	7
3.3 Thử sức mạnh của thuật toán ghép tối ưu	9
3.4 Thử sức mạnh của bảng băm	10
3.5 Tối ưu thứ tự mở rộng đỉnh	11
3.6 Mã nguồn Baby-4	12

Lời nói đầu

“Sokoban” là một trò chơi trí tuệ một người rất phổ biến. Nhiệm vụ của người chơi là điều khiển một công nhân trong nhà kho để đẩy N thùng giống nhau tới N ô đích giống nhau. Nếu chưa rõ luật chơi thì cũng không sao: chơi thử SokoMind trong phần đính kèm là biết. Tôi đã thêm vào đó 90 màn kiểm thử chuẩn, 61 màn trẻ em và 170 màn của Wenquxing. Trong phần giới thiệu về sau, ta sẽ dùng các màn ấy để kiểm thử chương trình, vì vậy tôi khuyên bạn nên tự thử trước xem mình vượt được bao nhiêu màn.

Vì nội dung bài viết không ít, trước hết tôi đưa ra một gợi ý nhỏ về cách đọc. Các bạn mới học, hoặc chưa biết thuật toán IDA*, nhất định nên đọc từ chương 1 và hiểu rõ nó, vì đó là nền tảng của toàn bài. Chương 2 rất dễ hiểu; chỉ cần nắm đại ý, biết khó khăn của bài toán Sokoban nằm ở đâu và vì sao ta chọn thuật toán IDA*. Bạn nào nóng lòng muốn xem chương trình có thể đọc trước chương 3: phiên bản đầu tiên S4-Baby của chúng ta ra đời ở đó, sau đó là một loạt biện pháp cải tiến. Những bạn thích trí tuệ nhân tạo cũng nên đọc kỹ, biết đâu sẽ tìm được cảm hứng. Cuối cùng là phần tổng kết, cũng quan trọng như chương 1, đừng bỏ qua.

Tôi giả định đọc giả của bài viết đã có hiểu biết nhất định về các kiến thức liên quan, nên thường không đưa ra định nghĩa hoặc chứng minh thật chặt chẽ, mà chỉ nhắc qua một cách thô lược và cố gắng dùng ngôn ngữ dễ hiểu. Nhưng vì trình độ của tôi thật sự có hạn, chắc chắn có không ít chỗ sai, rất mong mọi người phê bình và chỉ bảo.

Rất hoan nghênh mọi người liên hệ với tôi. Email: liurujia@163.net, OICQ: 2575127.

1 Kiến thức cơ bản về tìm kiếm không gian trạng thái

1.1 Không gian trạng thái

Đối với một bài toán thực tế, ta có thể trừu tượng hóa nó ở một mức độ nhất định. Nói một cách thông thường, trạng thái là mô tả toán học của tiến triển bài toán tại một thời điểm nào đó; chuyển trạng thái là thao tác đưa bài toán từ một trạng thái sang một, hoặc vài, trạng thái khác. Nếu chỉ có một tác tử có thể thực hiện các chuyển trạng thái ấy, thì mục tiêu của ta là đơn nhất: từ trạng thái bắt đầu xác định, qua một chuỗi chuyển trạng thái để đi tới một hoặc nhiều trạng thái đích.

Nếu có nhiều hơn một tác tử có thể điều khiển chuyển trạng thái, chẳng hạn trong đánh cờ, thì chúng có thể chuyển trạng thái theo những mục tiêu khác nhau, thậm chí đối lập nhau. Loại bài toán ấy không thuộc phạm vi thảo luận của bài viết này.

Ta biết rằng quá trình tìm kiếm thực chất là duyệt một đồ thị ẩn. Các đỉnh của nó là mọi trạng thái, còn các cạnh có hướng tương ứng với các chuyển trạng thái. Một lời giải khả thi là một đường đi từ đỉnh bắt đầu tới một đỉnh bất kỳ trong tập trạng thái đích. Đồ thị ấy gọi là không gian trạng thái; kiểu tìm kiếm như vậy gọi là tìm kiếm trong không gian trạng thái một tác tử.

1.2 Tìm kiếm mù

Tìm kiếm mù chủ yếu gồm các loại sau.

Tìm kiếm thuần ngẫu nhiên. Nghe có vẻ khá “ngốc”, nhưng khi độ sâu rất lớn, số lời giải khả thi nhiều và độ sâu lời giải lại không quan trọng, nó vẫn hữu dụng. Hơn nữa, tìm kiếm ngẫu nhiên sau khi cải tiến có thể xử lý các bài toán mà phân bố lời giải có quy luật nhất định, như tương đối dày đặc, tương

đồng đều, hoặc phân bố theo tỉ lệ vàng. Một ví dụ điển hình là khi bạn cuống lên tìm đồ, thường bạn đang thực hiện tìm kiếm ngẫu nhiên.

Tìm kiếm theo chiều rộng và chiều sâu. Mọi người hẳn đã quen thuộc với hiệu suất thời gian, hiệu suất không gian và đặc điểm của BFS và DFS. Ví dụ của tìm kiếm theo chiều rộng là khi kính của bạn rơi xuống đất và bạn bò trên sàn để tìm: bạn luôn sờ trước những chỗ gần mình nhất, nếu không thấy thì sờ xa hơn một chút. Ví dụ điển hình của tìm kiếm theo chiều sâu là đi trong mê cung. Chúng còn có các hình thức tìm kiếm ngược và hai chiều, nhưng những phần đó không nằm trong phạm vi thảo luận ở đây.

Tìm kiếm lặp. Các phép tìm kiếm này đặt một số hạn chế lên cách mở rộng cây tìm kiếm, rồi lặp lại việc tìm kiếm bằng cách nối dần điều kiện. Chúng gồm các phương pháp sau.

Lập sâu dần. Giới hạn độ sâu tối đa của cây tìm kiếm là $D_{\max}$, rồi tiến hành tìm kiếm. Nếu không có lời giải thì tăng $D_{\max}$ rồi tìm tiếp. Tuy làm nhiều công việc lặp lại, nhưng vì lượng việc tìm kiếm tăng theo hàm mũ của độ sâu, lượng việc ở lần trước, tức phần lặp lại, nhỏ hơn nhiều so với lượng tìm kiếm hiện tại. Phương pháp này thích hợp với các bài toán mà cây tìm kiếm nhìn chung vừa rộng vừa sâu, nhưng lời giải khả thi lại không quá sâu. DFS thông thường có thể sa vào một vùng rất sâu mà không có lời giải; còn BFS thì không đủ không gian.

Lập rộng dần. Phương pháp này giới hạn số con tối đa $B_{\max}$ được mở rộng từ một đỉnh. Nhưng vì ưu điểm không thật rõ, nó không được ứng dụng nhiều và cũng ít được nghiên cứu.

Tìm kiếm chùm. Phương pháp này giới hạn tổng số đỉnh tối đa $W_{\max}$ ở mỗi tầng của cây tìm kiếm. Rõ ràng khi đó kích thước cây tìm kiếm tỉ lệ thuận với độ sâu, nhưng có thể bỏ lỡ lời giải rất gần điểm bắt đầu. Khi tăng $W_{\max}$, việc giữ lại các đỉnh nào và tăng $W_{\max}$ bao nhiêu vẫn là vấn đề đang được nghiên cứu.

1.3 Tìm kiếm có thông tin

Ta cảm thấy một số bài toán rất “có cái để nghĩ”, chủ yếu vì chúng có nhiều thông tin gợi ý, dễ bắt đầu suy nghĩ, nhưng lại không dễ tìm được lời giải. Ta không muốn tự tay thử từng trường hợp một cách mù quáng, và cũng không muốn chương trình tìm kiếm một cách máy móc. Nói cách khác, ta hy vọng khai thác càng nhiều càng tốt đặc điểm riêng của bài toán để làm cho tìm kiếm thông minh hơn. Tìm kiếm có thông tin được giới thiệu dưới đây chính là một dạng tìm kiếm thông minh như vậy.

Trong các thuật toán vừa nhắc tới, ta chưa dùng thông tin của bản thân trạng thái, mà chỉ dùng chuyển trạng thái để tìm kiếm. Trên thực tế, khi tự giải bài, ta thường ước lượng xem một trạng thái còn cách mục tiêu bao xa, rồi dựa vào đó chọn giữa nhiều phương án. Đưa cách làm ấy vào tìm kiếm, ta có thể dùng một hàm đánh giá của trạng thái để ước lượng khoảng cách từ nó tới trạng thái đích. Hàm đánh giá này gắn rất chặt với bài toán và thể hiện một mức độ thông minh nhất định. Để tiện trình bày về sau, ta giới thiệu trước một số ký hiệu.

S	Một trạng thái bất kỳ của bài toán.
$H^*(s)$	Khoảng cách thực, ngắn nhất, từ s tới đích; tức là không biết trước.
$H(s)$	Hàm heuristic của s , tức cận dưới của khoảng cách từ s tới đích.
$G(s)$	Chi phí trước khi tới trạng thái s , thường lấy là độ sâu của s .
$F(s)$	Hàm đánh giá của s , tức ước lượng tổng chi phí để tới đích.

Với $H(s)$, ta yêu cầu $h(s) \leq h^*(s)$. Nếu với mọi trạng thái s_1 và s_2 , hàm h còn thỏa

$$h(s_1) \leq h(s_2) + c(s_1, s_2),$$

trong đó $c(s_1, s_2)$ là chi phí chuyển từ trạng thái s_1 sang s_2 , tức khi chuyển trạng thái thì mức giảm của cận dưới h nhiều nhất bằng chi phí thực của chuyển trạng thái, ta nói hàm h là nhất quán. Thực ra điều đó chỉ yêu cầu h không được giảm quá nhanh.

Trực giác cho ta

$$f(s) = g(s) + h(s),$$

tức tổng của chi phí đã trả và chi phí còn phải trả. Nếu h nhất quán, với s_1 và một đỉnh hậu duệ s_2 của nó ta có

$$h(s_1) \leq h(s_2) + c(s_1, s_2).$$

Cộng $g(s_1)$ vào hai vế, ta được

$$h(s_1) + g(s_1) \leq h(s_2) + g(s_1) + c(s_1, s_2),$$

tức $f(s_1) \leq f(s_2)$. Vì vậy hàm f tăng đơn điệu.

Tìm kiếm tham lam. Giống tìm kiếm theo chiều rộng, nó dùng một hàng đợi để lưu các đỉnh chờ mở rộng, nhưng sắp theo giá trị hàm h tăng dần, thực chất là hàng đợi ưu tiên. Rõ ràng vì h ước lượng không chính xác, tìm kiếm tham lam không bảo đảm lời giải đầu tiên tìm được là tối ưu, và có thể rất lâu cũng không tìm được lời giải nào.

Thuật toán A*. Thuật toán này rất giống tìm kiếm tham lam, nhưng sắp theo giá trị hàm f . Điều đó sinh thêm một vấn đề: trạng thái mới sinh ra có thể đã gặp trước đó. Vì sao? Tìm kiếm tham lam sắp theo hàm h , mà h chỉ liên quan tới trạng thái, nên không xuất hiện trùng lặp kiểu này; còn giá trị f không chỉ liên quan tới trạng thái, mà còn liên quan tới cách chuyển tới trạng thái s , nên cùng một trạng thái có thể có các giá trị f khác nhau. Cách giải quyết cũng đơn giản: nếu trạng thái mới s_1 giống một trạng thái s_2 đã gặp, giữ lại trạng thái có giá trị f nhỏ hơn. Nếu s_2 là đỉnh đang chờ mở rộng, vẫn có thể xảy ra $f(s_2) > f(s_1)$; chỉ các đỉnh đã mở rộng mới bảo đảm giá trị f tăng dần. A* bảo đảm tìm được lời giải tối ưu, nhưng dùng rất nhiều không gian, khó thích ứng với bài toán Sokoban của chúng ta.

Thuật toán IDA*. Vì A* có vấn đề về không gian, liệu ta có thể mượn ưu thế không gian của DFS, dùng phương thức tìm kiếm lặp để giảm áp lực hay không? Sau nghiên cứu, năm 1985 Korf đưa ra thuật toán Iterative Deepening A*, tức IDA*, giải quyết khá tốt vấn đề này. Ban đầu, ta đặt giá trị độ sâu tối đa $D_{\max}$ bằng giá trị h của đỉnh bắt đầu, rồi bắt đầu tìm kiếm theo chiều sâu, bỏ qua mọi đỉnh có giá trị f lớn hơn $D_{\max}$, nhờ đó giảm được rất nhiều lượng tìm kiếm. Nếu không có lời giải, lại tăng $D_{\max}$ cho tới khi tìm được một lời giải. Để chứng minh lời giải ấy nhất định là tối ưu. Vì đổi sang hình thức theo chiều sâu, so với A*, IDA* thực dụng hơn:

1. Không cần kiểm tra trùng, không cần sắp xếp, chỉ cần dùng ngăn xếp. Thao tác đơn giản.
2. Nhu cầu không gian giảm mạnh, có quan hệ logarit với kích thước cây tìm kiếm.

Các tìm kiếm có thông tin khác. Những phương pháp này gồm A* kiểu DFS cộng cắt tỉa tối ưu, A* hai chiều, v.v. Nhưng vì chúng chưa chín muồi hoặc không hữu dụng lắm, ở đây không giới thiệu thêm. A* còn có một dạng có trọng số; do trong bài toán Sokoban hiệu quả không rõ, ở đây cũng bỏ qua.

2 Bài toán Sokoban và đặc điểm của nó

Sau khi đã hiểu nhất định về các thuật toán tìm kiếm trong không gian trạng thái, ta xét bài toán Sokoban của mình. Dùng phương pháp nào thì tốt? Trước hết hãy xem đặc điểm của bài toán.

2.1 Bài toán Sokoban

Ở trên ta đã giới thiệu bài toán Sokoban. Ở đây tôi chỉ muốn nêu vài điểm cần chú ý liên quan tới việc giải bài. Trước hết, kích thước bản đồ Sokoban mà ta xét tối đa là 20×20 , đủ đáp ứng phần lớn các màn. Để tiện thảo luận về sau, ta đánh số bản đồ: các cột từ trái sang phải gọi là A, B, C, ..., còn các hàng từ trên xuống gọi là a, b, c, Vì việc người đi lại khi không đẩy thùng là không quan trọng, khi ghi lời giải ta bỏ qua vị trí và di chuyển của người, chỉ ghi các lần di chuyển của thùng. Hành động của người có thể suy ra rất dễ từ hành động của thùng.

Ghi chú về hình. Tại đây PDF đặt hình màn chuẩn số 1 kèm lời giải. Phần chữ trích xuất bằng pdftotext không chứa sơ đồ bản đồ đó.

Thế nào, ngay màn đầu đã cần nhiều bước như vậy. Các màn về sau sẽ ngày càng khó.

2.2 Đặc điểm của bài toán Sokoban

Trong lời nói đầu tôi đã nói khá lâu; tôi nghĩ dù trước đây bạn chưa chơi thì bây giờ cũng đã chơi thử rồi. Cảm giác thế nào? Có phải biến hóa quá nhiều, khó nắm bắt không? Không chỉ con người khó nắm bắt, ngay cả việc lập trình cũng trở nên khó hơn rất nhiều. Ta hãy so sánh nó với bài toán 8-puzzle kinh điển.

1. Bế tắc. Người mới chơi rất nhanh sẽ học được bế tắc là gì: chỉ cần đẩy một thùng vào góc. Rõ ràng bố cục như vậy chơi tiếp cũng hết hy vọng, vì về sau đẩy thế nào cũng không thể đưa thùng ấy ra khỏi góc. Không ít người chơi đã đúc kết nhiều kinh nghiệm về bế tắc, nhưng giải quyết vấn đề này một cách hệ thống không hề dễ. Ta sẽ dùng cả một chương, thực ra cũng không dài lắm, để phân tích vấn đề này.

##

#\$

Be tac dien hinh: thung bi ket trong goc tuong.

Ta lại nhìn bài toán 8-puzzle. Nó không có bế tắc, vì mỗi bước đều đảo ngược được. Ở điểm này, Sokoban đau đầu hơn nhiều. Dễ thấy không gian trạng thái như vậy không phải là đồ thị vô hướng, mà là đồ thị có hướng.

2. Không gian trạng thái. Bài toán 8-puzzle mỗi lần nhiều nhất có 4 cách di chuyển, số bước nhiều nhất cũng chỉ vài chục. Còn Sokoban thì sao? Với một màn khó, một bước có thể có hơn 100 lựa chọn, toàn bộ lời giải gồm hơn 600 động tác đẩy thùng. Cả hệ số phân nhánh lẫn độ sâu cây lời giải đều lớn như vậy, không gian trạng thái đương nhiên không nhỏ.

3. Ước lượng cận dưới. Trong tìm kiếm có thông tin, ta cần tính giá trị h , tức cần ước lượng cận dưới. Bài toán 8-puzzle có nhiều hàm cận dưới tốt, như tổng khoảng cách “về nhà”, nhưng Sokoban thì sao? Ta không thể trực tiếp tính khoảng cách “về nhà”, vì chưa rõ nhà của thùng nào là ô nào. Rất tự nhiên, ta có thể làm một phép ghép tối ưu trên đồ thị hai phía, nhưng cận dưới ấy thế nào?

a. Độ chính xác. Với A* và các biến thể của nó, cận dưới càng gần chi phí thực thì nói chung thuật toán càng hiệu quả. Phép ghép tối ưu của ta chỉ là “tình huống lý tưởng”. Trên thực tế, trong rất nhiều trường hợp các thùng ràng buộc lẫn nhau; việc buộc phải rời tuyến mục tiêu để nhường chỗ cho thùng khác là rất phổ biến. Chẳng hạn ở màn chuẩn số 50, có thùng phải đi qua ô đích rồi rời khỏi nó để nhường đường cho thùng khác. Hàm cận dưới của ta trả về 100, nhưng kết quả tốt nhất hiện có là 370. Chênh lệch thật lớn.

b. Hiệu suất. Vì hàm cận dưới được gọi rất thường xuyên, hiệu suất của nó không thể xem nhẹ. Độ phức tạp tiệm cận thời gian của ghép tối ưu khoảng $O(N^3)$, lớn hơn hàm cận dưới của 8-puzzle không biết bao nhiêu lần. Về sau ta sẽ đưa ra một vài cách cải tiến, nhưng bản chất của nó không thay đổi.

2.3 Giải bài toán Sokoban như thế nào

Đã có người chứng minh bài toán Sokoban là NP-hard, xem ra ta vẫn nên xét tìm kiếm. Nhớ lại tìm kiếm trong không gian trạng thái đã nhắc ở phần trước, dùng loại nào là tốt?

Vì đây là trò chơi trí tuệ, thông tin gợi ý có thể dùng rất phong phú. Ta không chỉ cần dùng, mà còn phải dùng càng đầy đủ càng tốt, nên cần dùng tìm kiếm có thông tin. Phía trên cũng đã nói, không gian trạng thái của Sokoban rất lớn, A* không còn cách nào, vì vậy ta chọn IDA*: hiện thực đơn giản và nhu cầu không gian cũng ít.

Nếu Sokoban khó như vậy, vì sao lại có nhiều người giải được khá nhiều màn? Các màn chuẩn 90 đã bị mọi người hiểu rất kỹ từ nhiều năm trước. Vì con người thông minh. Họ biết dự đoán, biết sắp xếp, biết học, có trực giác hỗ trợ và cũng có một tinh thần mạo hiểm nhất định. Họ, trong đó có cả tôi, thường dùng một số chiến lược giải bài ở “tầng cao”, vừa hiệu quả vừa linh hoạt. Srbga: muốn học không? Readers: đương nhiên muốn! Đáng tiếc các chiến lược ấy không đơn giản để học, cũng không thật có quy luật. Trong các chương sau, tôi sẽ cố hết sức mô phỏng cách tư duy của con người để thêm càng nhiều trí thông minh càng tốt cho chương trình của chúng ta.

3 Dùng thuật toán IDA* giải bài toán Sokoban: hiện thực và cải tiến

Trong phần trước, ta đã biết IDA* là thuật toán lõi để giải Sokoban. Trong phần này, ta sẽ dùng IDA* làm một chương trình giải Sokoban. Dù đây là phiên bản đầu tiên, gọi là S4-Baby, cũng đừng xem thường nó.

3.1 Khung thuật toán IDA*

Như đã nói, IDA* là thuật toán A* dựa trên DFS lặp, bỏ qua mọi đỉnh có giá trị f lớn hơn giới hạn độ sâu. Vì vậy, không khó viết giả mã khung của IDA*.

Giả mã 1: Khung thuật toán IDA*.

```
procedure IDA_STAR(StartState)
begin
  PathLimit := H( StartState ) - 1;
  Success := False;
  repeat
    inc(PathLimit);
    StartState.g := 0;
    Push(OpenStack, StartState);
```

```

repeat
  CurrentState:=Pop(OpenStack);
  If Solution(CurrentState) then
    Success = True
  Else if PathLimit >= CurrentState.g + H(CurrentState) then
    Foreach Child(CurrentState) do
      Push(OpenStack, Child(CurrentState));
    until Success or empty(OpenStack);
  until Success or ResourceLimitsReached;
end;

```

Đây chỉ là một khung rất thô, chưa làm được việc gì. Nhưng tôi nghĩ mọi người có thể đang nóng lòng muốn thử sức mạnh của IDA*, vì vậy ta cứ làm một chương trình cơ bản nhất trước.

3.2 Chương trình đầu tiên

Muốn từ khung biến thành chương trình cần điền thêm một số thứ. Ở đây ta lần lượt thảo luận.

Định dạng tệp vào và ra. Tệp vào là một tệp văn bản gồm N dòng, mỗi dòng là một số ký tự. Ý nghĩa của các ký tự như sau:

```

SPACE  o trong
.       o dich
$       thung
*       thung nam tren o dich
@       nguoi day
+       nguoi day nam tren o dich
#       tuong

```

Định dạng này nhất quán với định dạng của Xsokoban, SokoMind và Rolling Stone, nên khá tiện. Dòng đầu của tệp ra là số lần đẩy thùng M ; tiếp theo là M dòng, mỗi dòng có dạng $x \ y \ direction$, nghĩa là đẩy thùng ở hàng x , cột y thêm một bước theo hướng $direction$. Hướng có thể là một trong `left`, `right`, `up`, `down`, với $1 \leq x, y \leq 20$.

Cấu trúc dữ liệu. Vì đây là phiên bản đầu tiên, ta không cần nghĩ quá nhiều: chỉ cần chạy được và lập trình tiện là đủ, tạm thời không xét hiệu suất hay những thứ khác. Tối ưu là chuyện về sau.

Ta định nghĩa các kiểu dữ liệu mới `BitString`, `MazeType`, `MoveType`, `StateType` và `IDAType`. Xem chương trình trong phụ lục, không khó đoán ý nghĩa và công dụng của chúng. Điều duy nhất cần giải thích là kiểu `BitString`. Khi ghi trạng thái, ta xem bản đồ như một số lớn, mỗi ô là một bit. Khi đó toàn bộ các thùng tạo thành một `BitString`; kiểm tra một ô nào đó có thùng, ô đích hay tường hay không chỉ cần kiểm tra bit tương ứng có bằng 1 hay không. Cách này tuy lãng phí một ít không gian, nhưng phán đoán nhanh hơn và thao tác cũng đơn giản hơn.

Ta hợp nhất tọa độ x, y thành một biến `position`, trong đó

$$\text{Position} = (x - 1) \cdot \text{width} + (y - 1).$$

Ta dùng mảng hằng `DeltaPos:array[0..3]` để biểu diễn lượng tăng của `Position` khi đi lên, xuống, trái, phải.

Thuật toán. Để đơn giản, ta thậm chí chưa làm ghép tối ưu, mà dùng tổng khoảng cách từ mỗi thùng tới ô đích gần nhất làm hàm cận dưới. Nhưng “khoảng cách” ở đây là số lần đẩy. Khi tính bằng hàm MinPush, chỉ cần bỏ qua tất cả các thùng khác rồi dùng một lần BFS.

Hiệu quả. Hiệu quả này thì không nói cũng biết: không vượt được màn chuẩn nào. Nhưng để chứng minh chương trình là đúng, bạn có thể thử các màn trẻ em, tổng cộng 61 màn.

Gì cơ, màn đầu đã không thấy động tĩnh, sinh ra 180 nghìn đỉnh? Tuy vậy, rất nhiều màn vẫn qua rất nhanh. Ta lấy 1,000,000 đỉnh làm giới hạn, trên máy Celeron 300A của tôi chạy hơn mười phút, thu được kết quả kiểm thử sau.

No.	buoc	dinh	No.	buoc	dinh	No.	buoc	dinh
1	15	186476	21	8	102	41	11	145
2	6	24	22	7	110	42	10	118
3	5	14	23	10	192	43	12	223
4	6	24	24	10	432	44	8	63
5	9	31	25	4	23	45	12	138
6	5	8	26	11	846	46	14	178
7	6	35	27	3	18	47	8	296
8	11	39	28	9	38	48	8	156
9	4	12	29	10	142	49	5	60
10	5	14	30	8	641	50	11	14451
11	5	13	31	7	192	51	N/A	>1M
12	4	19	32	3	12	52	N/A	>1M
13	4	14	33	11	51	53	8	470
14	6	20	34	11	332	54	16	24270
15	6	57	35	16	11118	55	N/A	>1M
16	12	3947	36	10	242	56	14	3318
17	6	63	37	9	1171	57	N/A	>1M
18	11	5108	38	11	556	58	N/A	>1M
19	10	467	39	10	72	59	11	328
20	10	1681	40	9	203	60	N/A	>1M
						61	N/A	>1M

Những màn chưa giải được là 51, 52, 55, 57, 58, 60, 61. Những màn tương đối khó là 1, 16, 18, 20, 26, 30, 35, 37, 38, 50, 53, 54, 56. Dưới đây ta xem tình hình ước lượng cận dưới của các “màn khó”, để thấy cái giá phải trả cho việc “lười”.

man	so buoc toi uu	do sau ban dau	tong dinh	dinh tang tren
1	15	11	186476	7416
16	12	7	3947	844
18	11	10	5108	49
20	10	6	1681	42
26	11	5	846	394
30	8	6	641	200
35	16	3	11118	3464
37	9	4	1171	493
38	11	5	556	250
50	11	6	14451	51

53	8	5	470	48
54	16	9	24270	2562
56	14	4	3318	460

Có thể thấy ước lượng cận dưới liên quan rất lớn tới kích thước cây tìm kiếm. Hãy nhìn các màn 18, 20, 35, 50, 54, 56. Số đỉnh tầng trên ít biết bao. Nếu ngay từ đầu đã tìm từ tầng ấy thì... xem ra ta thật sự cần dùng thuật toán ghép tối ưu.

3.3 Thử sức mạnh của thuật toán ghép tối ưu

Được, bây giờ ta dùng thuật toán ghép tối ưu. Ghép tối ưu có thể hiện thực bằng luồng mạng, nhưng ở đây ta dùng thuật toán nhân đỉnh đã sửa đổi; tôi chép chương trình trong sách cho đỡ mất công. Bây giờ chương trình đổi tên thành Baby2.

Sau đây là tình hình kiểm thử của các “bài khó” vừa rồi.

man	buoc thuc	do sau dau	tong dinh Baby-1	tong dinh Baby-2
1	15	15	186476	60
16	12	10	3947	304
18	11	11	5108	46
20	10	8	1681	76
26	11	5	846	552
30	8	8	641	153
35	16	4	11118	6504
37	9	5	1171	438
38	11	5	556	546
50	11	7	14451	98
53	8	8	470	37
54	16	12	24270	273
56	14	4	3318	2225

Ồ, có trường hợp còn ít hơn cả số đỉnh tầng trên vừa rồi. Tất nhiên, cận dưới tốt hơn thì cắt tỉa theo độ sâu của tầng hiện tại cũng chính xác hơn.

Ngoài ra, bây giờ ta xem 12 màn đầu của Wenquxing. Các màn 1, 2, 4, 6, 8, 9, 11 đã có thể ra lời giải trong 50,000 đỉnh.

man	buoc thuc	do sau dau	tong dinh	dinh tang tren
1	31	31	75	75
2	11	11	142	142
4	26	18	33923	159
6	16	16	47	47
8	27	21	239	213
9	12	6	4806	2778
11	14	14	73	73

Vậy bước tiếp theo làm gì? In từng trạng thái ra phân tích, ta không khó phát hiện có rất nhiều đỉnh lặp. Vì vậy vấn đề tự nhiên tiếp theo là: làm sao tránh lặp?

3.4 Thử sức mạnh của bảng băm

Muốn kiểm tra trùng thì đương nhiên cần bảng băm. Nhưng có một vấn đề rất khó xử lý: biểu diễn trạng thái thế nào? Trong mở rộng đỉnh, ta dùng dòng bit để định nghĩa trạng thái của các thùng, nhưng ở đây thứ ta cần là chỉ số mảng phù hợp. Cách biểu diễn đó không thuận tiện. Vì vậy khi xây dựng bảng băm, ta dùng tọa độ các thùng để biểu diễn trạng thái, tức bộ N phần tử $(p[1], p[2], p[3], \dots, p[n])$. Còn hàm băm thì ta xét theo số mục của bảng băm. Ở đây, nếu số thùng nhiều nhất là 100, ta thử dùng 10,000 mục. Một phương án là cộng tất cả tọa độ lại, nhưng cách này có rất nhiều xung đột: vì khi thùng A dịch sang phải một ô, thùng B dịch sang trái một ô, giá trị hàm băm không đổi. Xét rằng cần giảm xung đột và hàm không nên quá phức tạp, ta dùng tổng có trọng số của tọa độ làm giá trị băm, tức $\sum k \cdot \text{Position}[k]$, dĩ nhiên cuối cùng lấy modulo 10,000. Thực ra hàm này cũng không tốt, nhưng tôi hơi lười, để sau cải tiến. Về xử lý xung đột, để đơn giản ta dùng phương pháp nối dây, lập danh sách liên kết để lưu mọi phần tử. Cần chú ý rằng hai trạng thái có cùng tọa độ thùng nhưng tọa độ người không liên thông là hai trạng thái khác nhau, đều phải lưu. Sau đây là kết quả kiểm thử các màn vừa rồi.

man	buoc thuc	do sau dau	dinh Baby2	dinh Baby3
Kid 1	15	15	60	52
Kid 16	12	10	304	189
Kid 18	11	11	46	41
Kid 20	10	8	76	67
Kid 26	11	5	552	192
Kid 30	8	8	153	145
Kid 35	16	4	6504	704
Kid 37	9	5	438	136
Kid 38	11	5	546	152
Kid 50	11	7	98	96
Kid 53	8	8	37	24
Kid 54	16	12	273	258
Kid 56	14	4	2225	1518
Wqx 1	31	31	75	75
Wqx 2	11	11	142	107
Wqx 4	26	18	33923	33916
Wqx 6	16	16	47	46
Wqx 8	27	21	239	226
Wqx 9	12	6	4806	968
Wqx 11	14	14	73	67

Các màn mới hoàn thành là:

man	buoc thuc	do sau dau	tong dinh	dinh tang tren
Kid 51	13	13	629	629
Kid 52	18	18	39841	39841
Kid 55	14	4	4886	1919
Kid 60	15	9	6916	916

Cần chú ý rằng khi chạy kid57 trong chế độ bảo vệ đã xuất hiện tràn heap, cho thấy việc lưu toàn bộ đỉnh là không cần thiết, vừa tốn không gian, vừa tốn thời gian, và cũng không mấy khả thi. Vậy ta nên làm gì? Ta biết đánh giá một bảng băm tốt hay xấu thường xét từ hai mặt: tần suất tra bảng thành công và lượng công việc tiết kiệm được sau khi tra thành công. Vì vậy ta có thể đặt hai bảng băm: một bảng

lưu các đỉnh gần đây, khả năng tra trùng lớn; một bảng lưu các đỉnh có độ sâu lớn, vì một khi tìm thấy thì tiết kiệm rất nhiều công việc. Làm như vậy không tăng bao nhiêu số đỉnh, nhưng giúp hiệu suất chương trình cao hơn và năng lực giải bài, tức khả năng chịu đựng về không gian, cũng cải thiện đáng kể. Tuy nhiên để tiện, chương trình của ta tạm thời chỉ dùng bảng thứ nhất.

3.5 Tối ưu thứ tự mở rộng đỉnh

Cải tiến cuối cùng trong phần này là tối ưu thứ tự mở rộng đỉnh. Mục tiêu không phải là cắt tĩa cây tìm kiếm, mà là hy vọng tìm được lời giải sớm hơn. Cách cải tiến cụ thể như sau:

1. Ưu tiên đẩy thùng vừa được đẩy.
2. Sau đó thử mọi phương án có thể làm giảm cận dưới; phương án giảm nhiều hơn được thử trước. Nếu mức giảm như nhau, đẩy thùng gần đích nhất trước.
3. Cuối cùng thử các phương án khác, cũng xét theo thứ tự như mục 2.

Có thể dự đoán rằng sau khi xử lý như vậy, việc tìm lời giải “tương đối dễ” sẽ xảy ra sớm hơn, nhưng cái giá do ước lượng cận dưới không chính xác gây ra không thể giảm, tức chỉ có thể giảm số đỉnh tăng trên. Tuy vậy, với tư cách một phương pháp cải tiến chuẩn của IDA*, ta vẫn cần thêm nó vào chương trình để thử.

Cần chú ý rằng ta dùng ngăn xếp, nên các phương án tương đối kém cần được đưa vào ngăn xếp trước. Kết quả kiểm thử thực tế cho thấy mục 1 hiệu quả khá tốt, còn mục 2 và 3 hiệu quả không tốt, thậm chí sinh nhiều đỉnh hơn. Nguyên nhân chủ yếu có thể là ước lượng cận dưới của ta không chính xác, trong khi mục 2 và 3 dùng tới hàm cận dưới. Trong phiên bản Baby-4 này, ta tắt hai biện pháp ở mục 2 và 3.

Được rồi, đã viết bốn phiên bản Baby, bạn có muốn so sánh không? Nhưng tôi chỉ quan tâm vài dữ liệu hơi khó.

man	buoc thuc	Baby-1	Baby-2	Baby-3	Baby-4
Kid 1	11	186476	60	52	38
Kid 16	7	3947	304	189	149
Kid 18	10	5108	46	41	31
Kid 35	16	11118	6504	704	462
Kid 50	11	14451	98	96	152
Kid 51	13	Too many	Too many	629	54
Kid 52	18	Too many	Too many	39841	97
Kid 54	16	24270	273	258	140
Kid 55	14	Too many	Too many	4886	3390
Kid 56	14	3318	2225	1518	1069
Kid 60	15	Too many	Too many	6916	5022
Wqx 4	26	97855	33923	33916	24251
Wqx 9	12	116927	4806	968	350

Từ bảng trên có thể thấy tối ưu của ta nhìn chung là có hiệu quả. Trực quan mà nói, nhiều trường hợp cải tiến không rõ là do ước lượng cận dưới tương đối kém; điểm này về sau ta sẽ tiếp tục thảo luận. Dù thế nào, vượt được 58 trong 61 màn “trẻ em” cũng khá tốt, ít nhất có thể nói phiên bản Baby của chương trình đã có “trí tuệ” của một trẻ em bình thường. Nhưng đây chỉ là mở đầu, phần hay còn ở phía sau.

3.6 Mã nguồn Baby-4

Chương trình S4BABY4.PAS nằm trong phần đính kèm; ở đây chỉ thêm một lượng nhỏ chú thích. Mọi người có thể thử hiệu quả của nó, nhưng không cần xem quá kỹ, vì trong các chương sau tôi sẽ sửa rất nhiều thứ, thậm chí khung chương trình chính của IDA* cũng sẽ trở nên khác.

Định nghĩa hằng.

```
const
{Version}
VerStr='S4 - SRbGa Super Sokoban Solver (Baby Version 4)';
Author='Written by Liu Rujia(SrbGa), 2001.2, Chongqing, China';

{Files}
InFile='soko.in';
OutFile='soko.out';

{Charactors}
Char_Soko='@';
Char_SokoInTarget='+';
Char_Box='$';
Char_BoxInTarget='*';
Char_Target='.';
Char_Wall='#';
Char_Empty=' ';

{Dimentions}
Maxx=21;
Maxy=21;
MaxBox=50;

{Directions}
Up=0;
Down=1;
Left=2;
Right=3;
DirectionWords:array[0..3] of string=('UP','DOWN','LEFT','RIGHT');

{Movement}
MaxPosition:integer=Maxx*Maxy;
Opposite:array[0..3] of integer=(1,0,3,2);
DeltaPos:array[0..3] of integer=(-Maxy,Maxy,-1,1);

Ta hợp nhất tọa độ x,y thành một giá trị position, trong đó  $position = (x - 1) \cdot maxy + (y - 1)$ .
Ở đây dùng hằng có kiểu vì về sau sẽ thay đổi giá trị MaxPosition theo kích thước bản đồ. Opposite
là hướng ngược, ví dụ Opposite[UP]:=DOWN; DeltaPos cũng sẽ được đặt lại. Khi di chuyển, ta chỉ cần
dùng NewPos:=OldPos+DeltaPos[Direction], rất tiện.

{IDA Related}
```

```

MaxNode=1000000;
MaxDepth=100;
MaxStack=150;
DispNode=1000;

```

Cứ sinh bao nhiêu đỉnh thì báo cáo một lần.

```

{HashTable}
MaxHashEntry=10000;
HashMask=10000;
MaxSubEntry=100;

{BitString}
BitMask:array[0..7] of byte=(1,2,4,8,16,32,64,128);

Infinite=Maxint;

```

Định nghĩa kiểu.

```

type
  PositionType=integer;

  BitString=array[0..Maxx*Maxy div 8-1] of byte;

```

Toàn bộ bản đồ là một BitString. Bit thứ position bằng 1 khi và chỉ khi vị trí position có thứ gì đó, như thùng, đích hoặc tường.

```

MapType=array[1..Maxx] of string[Maxy];
BiGraph=array[1..MaxBox,1..MaxBox] of integer;

```

```

MazeType=
record
  X,Y:integer;
  Map:MapType;
  GoalPosition:array[1..MaxBox] of integer;
  BoxCount:integer;
  Goals:BitString;
  Walls:BitString;
end;

```

Các trường lần lượt là kích thước, dữ liệu gốc dùng để hiển thị trạng thái, BitString của các ô đích, tổng số thùng, vị trí đích, và BitString của tường. Cả BitString lẫn mảng vị trí đều được dùng để tăng tốc.

```

MoveType=
record
  Position:integer;
  Direction:0..3;
end;

```

Direction là hướng mà thùng bị đẩy tới.

```
StateType=
record
  Boxes:BitString;
  ManPosition:PositionType;
  MoveCount:integer;
  Move:array[1..MaxDepth] of MoveType;
  g,h:integer;
end;

IDAType=
record
  TopLevelNodeCount:longint;
  NodeCount:longint;
  StartState:StateType;
  PathLimit:integer;
  Top:integer;
  Stack:array[1..MaxStack] of StateType;
end;
```

Top là con trỏ đỉnh ngăn xếp.

```
PHashTableEntry=^HashTableEntry;
HashTableEntry=
record
  Next:PHashTableEntry;
  State:StateType;
end;

PHashTableType=^HashTableType;
HashTableType=
record
  FirstEntry:array[0..MaxHashEntry] of PHashTableEntry;
  Count:array[0..MaxHashEntry] of byte;
end;
```

Đây là các kiểu liên quan tới bảng băm. Ta dùng phương pháp nối dây, nhờ đó có thể xin không gian heap bằng con trỏ; kết hợp với chế độ bảo vệ thì hiệu quả tốt hơn.

```
var
  HashTable:PHashTableType;
  SokoMaze:MazeType;
  IDA:IDAType;

procedure SetBit(var BS:BitString; p:integer);
begin
  BS[p div 8]:=BS[p div 8] or BitMask[p mod 8];
end;
```

```

procedure ClearBit(var BS:BitString; p:integer);
begin
  BS[p div 8]:=BS[p div 8] xor BitMask[p mod 8];
end;

function GetBit(var BS:BitString; p:integer):byte;
begin
  if BS[p div 8] and BitMask[p mod 8]>0 then GetBit:=1 else GetBit:=0;
end;

```

Đây là các thao tác bit: đặt, xóa và lấy một mục của BitString.

```

procedure Init;
var
  Lines:MapType;

procedure ReadInputFile;
var
  f:text;
  s:string;
begin
  SokoMaze.X:=0;
  SokoMaze.Y:=0;
  SokoMaze.BoxCount:=0;
  assign(f,infile);
  reset(f);
  while not eof(f) do
  begin
    readln(f,s);
    if length(s)>SokoMaze.Y then
      SokoMaze.Y:=length(s);
    inc(SokoMaze.X);
    Lines[SokoMaze.X]:=s;
  end;
  close(f);
end;

procedure AdjustData;
var
  i,j:integer;
begin
  for i:=1 to SokoMaze.X do
    while length(Lines[i])<SokoMaze.Y do
      Lines[i]:=Lines[i]+' ';

  SokoMaze.Map:=Lines;
  for i:=1 to SokoMaze.X do
    for j:=1 to SokoMaze.Y do

```

```

    if SokoMaze.Map[i,j] in [Char_BoxInTarget,Char_SokoInTarget,Char_Target] then
        SokoMaze.Map[i,j]:=Char_Target
    else if SokoMaze.Map[i,j]<>Char_Wall then
        SokoMaze.Map[i,j]:=Char_Empty;

```

Điều chỉnh mảng Map, bỏ thùng và người đẩy ra.

```

for i:=1 to SokoMaze.X do
    for j:=1 to SokoMaze.Y do
        if Lines[i,j] in [Char_Target,Char_BoxInTarget,Char_SokoInTarget] then
            begin
                inc(SokoMaze.BoxCount);
                SokoMaze.GoalPosition[SokoMaze.BoxCount]:=(i-1)*SokoMaze.Y+j-1;
            end;

```

Thống kê số ô đích và GoalPosition.

```

DeltaPos[Up]:=-SokoMaze.Y;
DeltaPos[Down]:=SokoMaze.Y;
MaxPosition:=SokoMaze.X*SokoMaze.Y;

```

Điều chỉnh DeltaPos và MaxPosition theo kích thước bản đồ.

```

end;

```

```

procedure ConstructMaze;
var
    i,j:integer;
begin
    fillchar(SokoMaze.Goals,sizeof(SokoMaze.Goals),0);
    fillchar(SokoMaze.Walls,sizeof(SokoMaze.Walls),0);

    for i:=1 to SokoMaze.X do
        for j:=1 to SokoMaze.Y do
            case Lines[i,j] of
                Char_SokoInTarget, Char_BoxInTarget, Char_Target:
                    SetBit(SokoMaze.Goals,(i-1)*SokoMaze.Y+j-1);
                Char_Wall:
                    SetBit(SokoMaze.Walls,(i-1)*SokoMaze.Y+j-1);
            end;
        end;
    end;
end;

```

```

procedure InitIDA;
var
    i,j:integer;
    StartState:StateType;
begin
    IDA.NodeCount:=0;
    IDA.TopLevelNodeCount:=0;

```



```

fillchar(StartState,sizeof(StartState),0);

for i:=1 to SokoMaze.X do
  for j:=1 to SokoMaze.Y do
    case Lines[i,j] of
      Char_Soko, Char_SokoInTarget:
        StartState.ManPosition:=(i-1)*SokoMaze.Y+j-1;
      Char_Box, Char_BoxInTarget:
        SetBit(StartState.Boxes,(i-1)*SokoMaze.Y+j-1);
    end;

  StartState.g:=0;
  IDA.StartState:=StartState;
  new(HashTable);
  for i:=1 to MaxHashEntry do
    begin
      HashTable^.FirstEntry[i]:=nil;
      HashTable^.Count[i]:=0;
    end;
  end;

begin
  ReadInputFile;
  AdjustData;
  ConstructMaze;
  InitIDA;
end;

procedure PrintState(State:StateType);
var
  i,x,y:integer;
  Map:MapType;
begin
  Map:=SokoMaze.Map;
  x:=State.ManPosition div SokoMaze.Y+1;
  y:=State.ManPosition mod SokoMaze.Y+1;
  if Map[x,y]=Char_Target then
    Map[x,y]:=Char_SokoInTarget
  else
    Map[x,y]:=Char_Soko;

  for i:=1 to MaxPosition do
    if GetBit(State.Boxes,i)>0 then
      begin
        x:=i div SokoMaze.Y+1;
        y:=i mod SokoMaze.Y+1;
        if Map[x,y]=Char_Target then
          Map[x,y]:=Char_BoxInTarget

```

```

        else
            Map[x,y]:=Char_Box;
        end;
    for i:=1 to SokoMaze.X do
        Writeln(Map[i]);
    end;

function Solution(State:StateType):boolean;
var
    i:integer;
begin
    Solution:=false;
    for i:=1 to MaxPosition do
        if (GetBit(State.Boxes,i)>0) and (GetBit(SokoMaze.Goals,i)=0) then
            exit;
        Solution:=true;
    end;

function CanReach(State:StateType; Position:integer):boolean;

    Dùng BFS để xét trong trạng thái State, người đẩy có thể tới Position hay không.

var
    Direction:integer;
    Pos,NewPos:integer;
    Get,Put:integer;
    Queue:array[0..Maxx*Maxy] of integer;
    Reached:Array[0..Maxx*Maxy] of boolean;
begin
    fillchar(Reached,sizeof(Reached),0);

    Pos:=State.ManPosition;
    Get:=0; Put:=1;
    Queue[0]:=Pos;
    Reached[Pos]:=true;

    CanReach:=true;
    while Get<>Put do
    begin
        Pos:=Queue[Get];
        inc(Get);
        if Pos=Position then
            exit;
        for Direction:=0 to 3 do
            begin
                NewPos:=Pos+DeltaPos[Direction];
                if Reached[NewPos] then continue;
                if GetBit(State.Boxes,NewPos)>0 then continue;
                if GetBit(SokoMaze.Walls,NewPos)>0 then continue;
            end;
        end;
        Queue[Put]:=NewPos;
        Reached[NewPos]:=true;
        inc(Put);
    end;
end;

```

```

    Reached[NewPos]:=true;
    Queue[Put]:=NewPos;
    inc(Put);
end;
end;
CanReach:=false;
end;

```

```

function MinPush(BoxPosition,GoalPosition:integer):integer;

```

Khi không có thùng khác, từ BoxPosition đẩy tới GoalPosition cần ít nhất bao nhiêu bước.

```

var
  i:integer;
  Direction:integer;
  Pos,NewPos,ManPos:integer;
  Get,Put:integer;
  Queue:array[0..Maxx*Maxy] of integer;
  Distance:Array[0..Maxx*Maxy] of integer;
begin
  for i:=0 to Maxx*Maxy do
    Distance[i]:=Infinite;

  Pos:=BoxPosition;
  Get:=0; Put:=1;
  Queue[0]:=Pos;
  Distance[Pos]:=0;

  while Get<>Put do
  begin
    Pos:=Queue[Get];
    inc(Get);
    if Pos=GoalPosition then
    begin
      MinPush:=Distance[Pos];
      exit;
    end;
    for Direction:=0 to 3 do
    begin
      NewPos:=Pos+DeltaPos[Direction];
      ManPos:=Pos+DeltaPos[Opposite[Direction]];
      {Nguoi phai dung o phia sau}
      if Distance[NewPos]<Infinite then continue;
      if GetBit(SokoMaze.Walls,NewPos)>0 then continue;
      {Khong day duoc}
      if GetBit(SokoMaze.Walls,ManPos)>0 then continue;
      {Nguoi khong co cho dung}
      Distance[NewPos]:=Distance[Pos]+1;
      Queue[Put]:=NewPos;
    end;
  end;
end;

```

```

        inc(Put);
    end;
end;
MinPush:=Infinite;
end;

procedure DoMove(State:StateType; Position,Direction:integer;
                var NewState:StateType);
var
    NewPos:integer;
begin
    NewState:=State;
    NewPos:=Position+DeltaPos[Direction];
    NewState.ManPosition:=Position;
    SetBit(NewState.Boxes,NewPos);
    ClearBit(NewState.Boxes,Position);
end;

function MinMatch(BoxCount:integer;Gr:BiGraph):integer;

```

Đây là thuật toán chuẩn, chép từ chương trình trong sách, không cần xem kỹ.

```

var
    VeryBig:integer;
    TempGr:BiGraph;
    L:array[1..MaxBox*2] of integer;
    SetX,SetY,MatchedX,MatchedY:Set of 1..MaxBox;

procedure MaxMatch(n,m:integer);
function Path(x:integer):boolean;
var
    i,j:integer;
begin
    Path:=false;
    for i:=1 to m do
        if not (i in SetY)and(Gr[x,i]<>0) then
            begin
                SetY:=SetY+[i];
                if not (i in MatchedY) then
                    begin
                        Gr[x,i]:=-Gr[x,i];
                        MatchedY:=MatchedY+[i];
                        Path:=true;
                        exit;
                    end;
                j:=1;
                while (j<=m)and not (j in SetX) and (Gr[j,i]>=0) do inc(j);
                if j<=m then
                    begin

```

```

        SetX:=SetX+[j];
        if Path(j) then
        begin
            Gr[x,i]:=-Gr[x,i];
            Gr[j,i]:=-Gr[j,i];
            Path:=true;
            exit;
        end;
    end;
end;
end;

var
    u,i,j,al:integer;

begin
    Fillchar(L,sizeof(L),0);
    TempGr:=Gr;
    for i:=1 to n do
        for j:=1 to m do
            if L[i]<Gr[i,j] then
                L[i]:=Gr[i,j];
    u:=1; MatchedX:=[]; MatchedY:=[];
    for i:=1 to n do
        for j:=1 to m do
            if L[i]+L[n+j]=TempGr[i,j] then
                Gr[i,j]:=1
            else
                Gr[i,j]:=0;
    while u<=n do
    begin
        SetX:=[u]; SetY:=[];
        if not (u in MatchedX) then
        begin
            if not Path(u) then
            begin
                al:=Infinite;
                for i:=1 to n do
                    for j:=1 to m do
                        if (i in SetX) and not (j in SetY) and (L[i]+L[n+j]-TempGr[i,j]<al) then
                            al:=L[i]+L[n+j]-TempGr[i,j];
                for i:=1 to n do if i in SetX then L[i]:=L[i]-al;
                for i:=1 to m do if i in SetY then l[n+i]:=l[n+i]+al;
                for i:=1 to n do
                    for j:=1 to m do
                        if l[i]+l[n+j]=TempGr[i,j] then
                            Gr[i,j]:=1
                        else

```

```

        Gr[i,j]:=0;
    MatchedX:=[]; MatchedY:=[];
    for i:=1 to n+m do
        if l[i]<-1000 then
            exit;
        end
        else
            MatchedX:=MatchedX+[u];
            u:=0;
        end;
        inc(u);
    end;
end;

var
    i,j:integer;
    Tot:integer;
begin
    VeryBig:=0;
    for i:=1 to BoxCount do
        for j:=1 to BoxCount do
            if (Gr[i,j]<Infinite)and(Gr[i,j]>VeryBig) then
                VeryBig:=Gr[i,j];
            inc(VeryBig);

        for i:=1 to BoxCount do
            for j:=1 to BoxCount do
                if Gr[i,j]<Infinite then
                    Gr[i,j]:=VeryBig-Gr[i,j]
                else
                    Gr[i,j]:=0;

```

Các câu lệnh này thực hiện phép chuyển sang bài toán bổ sung.

```

MaxMatch(BoxCount,BoxCount);
Tot:=0;
for i:=1 to BoxCount do
begin
    for j:=1 to BoxCount do
        if Gr[i,j]<0 then
            begin
                Tot:=Tot+VeryBig-TempGr[i,j];
                break;
            end;
        if Gr[i,j]>=0 then
            begin
                MinMatch:=Infinite;
                exit;
            end;

```

```

end;
MinMatch:=Tot;
end;

function CalcHeuristicFunction(State:StateType):integer;

    Tính giá trị hàm heuristic.

var
    H,Min:integer;
    i,j,p,Count,BoxCount,Cost:integer;
    BoxPos:array[1..MaxBox] of integer;
    Distance:BiGraph;
begin
    p:=0;
    for i:=1 to MaxPosition do
        if GetBit(State.Boxes,i)>0 then
            begin
                inc(p);
                BoxPos[p]:=i;
            end;
        for i:=1 to p do
            for j:=1 to p do
                Distance[i,j]:=MinPush(BoxPos[i],SokoMaze.GoalPosition[j]);
            end;
        end;
    end;
    BoxCount:=SokoMaze.BoxCount;

    H:=0;
    for i:=1 to BoxCount do
        begin
            Count:=0;
            for j:=1 to BoxCount do
                if Distance[i,j]<Infinite then
                    inc(Count);
                end;
            end;
            if Count=0 then
                {Co mot thung khong day toi bat ky dich nao}
                begin
                    CalcHeuristicFunction:=Infinite;
                    exit;
                end;
            end;
        end;
    end;

    H:=MinMatch(BoxCount, Distance);
    CalcHeuristicFunction:=H;
end;

function HashFunction(State:StateType):integer;
var
    i,h,p:integer;
begin

```

```

h:=0;
p:=0;
for i:=1 to MaxPosition do
  if GetBit(State.Boxes,i)>0 then
    begin
      inc(p);
      h:=(h+p*i) mod HashMask;
      {Ban co the tu doi ham nay}
    end;
  HashFunction:=h;
end;

function SameState(S1,S2:StateType):boolean;
var
  i:integer;
begin
  SameState:=false;
  for i:=1 to MaxPosition do
    if GetBit(S1.Boxes,i)<>GetBit(S2.Boxes,i) then
      exit;
  if not CanReach(S1,S2.ManPosition) then
    {Chi can hai vi tri nguoi lien thong thi nen xem la cung mot trang thai}
    exit;
  SameState:=true;
end;

function Prior(State:StateType;M1,M2:MoveType):boolean;
var
  NewPos:integer;
  Inertia1,Inertia2:boolean;
  S1,S2:StateType;
  H1,H2:integer;
begin
  Prior:=false;
  if State.MoveCount>0 then
    begin
      NewPos:=State.Move[State.MoveCount].Position+
        DeltaPos[State.Move[State.MoveCount].Direction];
      if NewPos=M1.Position then Inertia1:=true else Inertia1:=false;
      {Uu tien thao tac day lien tiep cung mot thung}
      if NewPos=M2.Position then Inertia2:=true else Inertia2:=false;
      if Inertia1 and not Inertia2 then begin Prior:=true; exit; end;
      if Inertia2 and not Inertia1 then begin Prior:=false; exit; end;
    end;
end;

procedure IDA_Star;
var

```



```

Sucess:boolean;
CurrentState:StateType;
H:integer;
f:Text;

procedure IDA_Push(State:StateType);
begin
  if IDA.Top=MaxStack then
    Exit;
  inc(IDA.Top);
  IDA.Stack[IDA.Top]:=State;
end;

procedure IDA_Pop(var State:StateType);
begin
  State:=IDA.Stack[IDA.Top];
  dec(IDA.Top);
end;

function IDA_Empty:boolean;
begin
  IDA_Empty:=(IDA.Top=0);
end;

```

Trên đây là các thao tác ngăn xếp.

```

procedure IDA_AddToHashTable(State:StateType);
var
  h:integer;
  p:PHashTableEntry;
begin
  h:=HashFunction(State);
  if HashTable^.Count[h]<MaxSubEntry then
    begin
      new(p);
      p^.State:=State;
      p^.Next:=HashTable^.FirstEntry[h];
      HashTable^.FirstEntry[h]:=p;
      inc(HashTable^.Count[h]);
    end
  else begin
    p:=HashTable^.FirstEntry[h];
    while p^.Next^.Next<>nil do
      p:=p^.Next;
    p^.Next^.State:=State;
    p^.Next^.Next:=HashTable^.FirstEntry[h];
    HashTable^.FirstEntry[h]:=p^.Next;
    p^.Next:=nil;
  end;
end;

```

```

end;

function IDA_InHashTable(State:StateType):boolean;
var
  h:integer;
  p:PHashTableEntry;
begin
  h:=HashFunction(State);
  p:=HashTable^.FirstEntry[h];
  IDA_InHashTable:=true;
  while p<>nil do
  begin
    if SameState(p^.State,State) then
    begin
      if p^.State.g>State.g then
      begin
        p^.State.g:=State.g;
        IDA_InHashTable:=false;

```

Nếu mục tìm thấy trong bảng có độ sâu lớn hơn, điều đó không có nghĩa là lần này với độ sâu nhỏ hơn cũng vô nghiệm. Đúng ra nên cập nhật động cận dưới; ở đây ta xử lý như chưa tìm thấy. Các chương sau sẽ cải tiến chỗ này.

```

      end;
      exit;
    end;
    p:=p^.Next;
  end;
  IDA_InHashTable:=false;
end;

```

Đó là các thao tác bảng băm.

```

procedure IDA_AddNode(State:StateType);
begin
  IDA_Push(State);
  inc(IDA.NodeCount);
  if IDA.NodeCount mod DispNode=0 then
    Writeln('NodeCount=',IDA.NodeCount);
  inc(IDA.TopLevelNodeCount);
  IDA_AddToHashTable(State);
end;

procedure IDA_Expand(State:StateType);
var
  MoveCount:integer;
  MoveList:array[1..Maxx*Maxy*4] of MoveType;
  t:MoveType;
  i,j,Direction:integer;

```

```

NewBoxPos, NewManPos:integer;
NewState:StateType;
begin
MoveCount:=0;
for i:=1 to MaxPosition do
  if GetBit(State.Boxes,i)>0 then
    for Direction:=0 to 3 do
      begin
        NewBoxPos:=i+DeltaPos[Direction];
        NewManPos:=i+DeltaPos[Opposite[Direction]];
        if GetBit(State.Boxes,NewBoxPos)>0 then continue;
        if GetBit(SokoMaze.Walls,NewBoxPos)>0 then continue;
        if GetBit(State.Boxes,NewManPos)>0 then continue;
        if GetBit(SokoMaze.Walls,NewManPos)>0 then continue;
        if CanReach(State,NewManPos) then
          begin
            DoMove(State,i,Direction,NewState);
            if CalcHeuristicFunction(NewState)=Infinite then continue;
            if CalcHeuristicFunction(NewState)+State.g>=IDA.PathLimit then continue;
            {Cot loi cua IDA*: gioi han do sau}
            if IDA_InHashTable(NewState) then continue;
            inc(MoveCount);
            MoveList[MoveCount].Position:=i;
            MoveList[MoveCount].Direction:=Direction;
          end;
        end;
      end;
    end;

for i:=1 to MoveCount do
  for j:=i+1 to MoveCount do
    if Prior(State,MoveList[i],MoveList[j]) then
      {Dieu chinh thu tu cach day}
      begin
        t:=MoveList[j];
        MoveList[j]:=MoveList[i];
        MoveList[i]:=t;
      end;
    end;

for i:=1 to MoveCount do
  {Lan luot xet moi phuong an di chuyen}
  begin
    DoMove(State,MoveList[i].Position,MoveList[i].Direction,NewState);
    inc(NewState.MoveCount);
    NewState.Move[NewState.MoveCount].Position:=MoveList[i].Position;
    NewState.Move[NewState.MoveCount].Direction:=MoveList[i].Direction;
    NewState.g:=State.g+1;
    IDA_AddNode(NewState);
  end;
end;
end;

```

```

procedure IDA_Answer(State:StateType);
var
  i:integer;
  x,y:integer;
begin
  Writeln(f,'Solution Found in ', State.MoveCount,' Pushes');
  for i:=1 to State.Movecount do
  begin
    x:=State.Move[i].Position div SokoMaze.Y+1;
    y:=State.Move[i].Position mod SokoMaze.Y+1;
    Writeln(f, x,' ',y,' ',DirectionWords[State.Move[i].Direction]);
  end;
end;

begin
  Writeln(VerStr);
  Writeln(Author);

  IDA.PathLimit:=CalcHeuristicFunction(IDA.StartState)-1;
  Sucess:=false;
  repeat
    inc(IDA.PathLimit);
    Writeln('Pathlimit=',IDA.PathLimit);
    IDA.TopLevelNodeCount:=0;
    IDA.Top:=0;
    IDA.StartState.g:=0;
    IDA_Push(IDA.StartState);
    repeat
      IDA_Pop(CurrentState);
      H:=CalcHeuristicFunction(CurrentState);
      if H=Infinite then continue;
      if Solution(CurrentState) then
        Sucess:=true
      else if IDA.PathLimit>=CurrentState.g+H then
        IDA_Expand(CurrentState);
    until Sucess or IDA_Empty or (IDA.NodeCount>MaxNode);
    Writeln('PathLimit ',IDA.PathLimit,' Finished. NodeCount=',IDA.NodeCount);
  until Sucess or (IDA.PathLimit>=MaxDepth) or (IDA.NodeCount>MaxNode);

  Assign(f,outfile);
  ReWrite(f);
  Writeln(f,VerStr);
  Writeln(f,Author);
  Writeln(f);

  if not Sucess then
    Writeln(f,'Cannot find a solution.')

```

```
else
    IDA_Answer(CurrentState);

Writeln('Node Count:',IDA.NodeCount);
Writeln;
close(f);
end;

begin
    Init;
    IDA_Star;
end.
```

Ghi chú về bản trích xuất

Tệp PDF có tên “incomplete” và phần văn bản trích được bằng `pdftotext -layout -enc UTF-8` kết thúc sau mã nguồn Baby-4 ở trên. Các mục tiếp theo trong mục lục và các phụ lục chỉ xuất hiện dưới dạng tiêu đề trong phần đầu của bản trích xuất.